



Pimpri Chinchwad Education Trust's
**Pimpri Chinchwad College of Engineering
(PCCoE)**
(An Autonomous Institute)
Affiliated to Savitribai Phule Pune
University(SPPU) ISO 21001:2018 Certified by
TUV



Course: Software Testing and Quality Assurance Laboratory	Code: BIT26PE11
Name: Amar Vaijinath Chavan	PRN: 124B2F001
Assignment 5: Implement an automated test using Selenium WebDriver and WebDriver Manager in Python/Java for a sample or real website, simulating a given scenario and verifying its success.	

1. OBJECTIVE

- To design and implement automated **System Test cases** for the "Hotel Management System" login module.
- To utilize **Selenium WebDriver** for simulating real-world user interactions within a web browser.
- To verify the end-to-end integrity of the **User Authentication** flow using Python-based automation scripts.

2. TEST ENVIRONMENT & CONFIGURATION

2.1 Software Stack

- **Operating System:** Windows 11 (XAMPP Environment)
- **Language:** Python 3.11
- **Automation Tool:** Selenium WebDriver 4
- **Driver Management:** WebDriver Manager for Python

2.2 Browser Automation Configuration

The automation script serves as the test runner, defining the browser lifecycle and interaction synchronization.

- **Service:** Uses ChromeDriverManager to automatically resolve and install the correct chromedriver.exe.
- **Implicit/Explicit Waits:** Ensures elements like Bootstrap Modals are fully rendered before interaction.
- **Window Management:** Automatically maximizes the browser to ensure all UI elements are interactable.

3. PROJECT STRUCTURE

Project Root: C:\xampp\htdocs\hbwebsite

- **index.php** — The Landing Page containing the Login/Register Modals (System Under Test).
- **Root folder**— Python scripts containing Selenium test cases.
- **inc/header.php** — Contains the UI triggers for the Login Modal.
- **ajax/login_register.php** — The backend target for authentication requests.

4. IMPLEMENTATION DETAILS (AUTOMATION LOGIC)

The automation logic focuses on the **Black-box Testing** of the system, interacting with the frontend precisely as a user would. This ensures that the integration between the UI, JavaScript, and PHP backend is seamless.

4.1 Browser & Session Initialization

This module handles the launching of the Chrome browser and navigating to the local server environment.

- **Description:** Logic for environment setup, window maximizing, and navigating to the hbwebsite root URL.

4.2 Modal Interactivity & Synchronization

Logic for triggering the Bootstrap Login Modal and managing the timing of animations.

- **Description:** Implementation of WebDriverWait to handle the asynchronous "fade-in" of the Login Modal.

4.3 Element Location & Data Entry

Uses advanced locators (XPath/Name) to identify specific input fields within the modal container.

- **Description:** Targeting and populating the email_mob and pass fields with valid test credentials.

4.4 Form Submission & Event Handling

Simulates the clicking of the "LOGIN" submit button to trigger the AJAX authentication request.

- **Description:** Logic for identifying the submit button and executing the click action to send data to the server.

4.5 Session Verification & Assertion

The final logic to determine if the login was successful by checking the state of the post-login UI.

- **Description:** Automated check for the "Logout" button in the page source to verify a successful PHP session start.

5. SYSTEM TEST SUITE DEVELOPMENT

Test cases were developed using the **Selenium Python API**. These tests target the frontend components to verify that user-side authentication requirements are met.

5.1 Positive Login Test Case

Testing the successful journey of a registered user using valid credentials.

- **Description:** Selenium test methods utilizing `EC.element_to_be_clickable` to verify the primary login requirement.

5.2 Synchronization & Timing Validation

Validation of the script's ability to handle network latency and local server response times.

- **Description:** Implementation of `time.sleep()` and Explicit Waits to ensure test stability on a local XAMPP server.

6. TEST EXECUTION & REPORTING

The test suite was executed via the **VS Code Terminal** to provide real-time feedback on the browser's state and the final test outcome.

```
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from webdriver_manager.chrome import ChromeDriverManager
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import time

# Credentials provided by user
USER_EMAIL = "Test001@gmail.com"
USER_PASS = "123456"

# Initialize WebDriver
service = Service(ChromeDriverManager().install())
driver = webdriver.Chrome(service=service)

try:
    # Step 1: Navigate to the Hotel Website
    driver.get("http://localhost/hbwebsite")
    driver.maximize_window()
    wait = WebDriverWait(driver, 10)
    time.sleep(2) # For screenshot: Home Page

    # Step 2: Trigger the Login Modal
    print("Opening Login Modal...")
    login_trigger = wait.until(EC.element_to_be_clickable((By.XPATH, "//button[contains(text(), 'Login')]")))
    login_trigger.click()

    # Wait for the modal to be visible
    wait.until(EC.visibility_of_element_located((By.ID, "loginModal")))
    time.sleep(2) # For screenshot: Login Modal Visible

    # Step 3: Enter Existing Credentials
    print(f"Entering credentials for {USER_EMAIL}...")
```

```

driver.find_element(By.XPATH,
"//div[@id='loginModal']/input[@name='email_mob']").send_keys(USER_EMAIL)
time.sleep(1)
driver.find_element(By.XPATH,
"//div[@id='loginModal']/input[@name='pass']").send_keys(USER_PASS)
time.sleep(1) # For screenshot: Credentials filled

# Step 4: Submit the Form
submit_btn = driver.find_element(By.XPATH, "//div[@id='loginModal']/button[@type='submit']")
submit_btn.click()
print("Login submitted.")

# Step 5: Final Verification
time.sleep(5) # Wait for page refresh and session creation
if "Logout" in driver.page_source:
    print("Test Result: LOGIN SUCCESSFUL - PASS")
else:
    print("Test Result: LOGIN FAILED - Check credentials or Server response.")

finally:
    # Keep browser open for a few seconds to verify visually
    time.sleep(3)
    driver.quit()

```

6.1 Execution Command

The following command was used to trigger the automated Selenium script: `python selenium_login_test.py`

```

[Running] python -u "c:\xampp\htdocs\hbwebsite\seleniumtestRegisterLogin.py"
Opening Login Modal...
Entering credentials for Test001@gmail.com...
Login submitted.
Test Result: LOGIN SUCCESSFUL - PASS

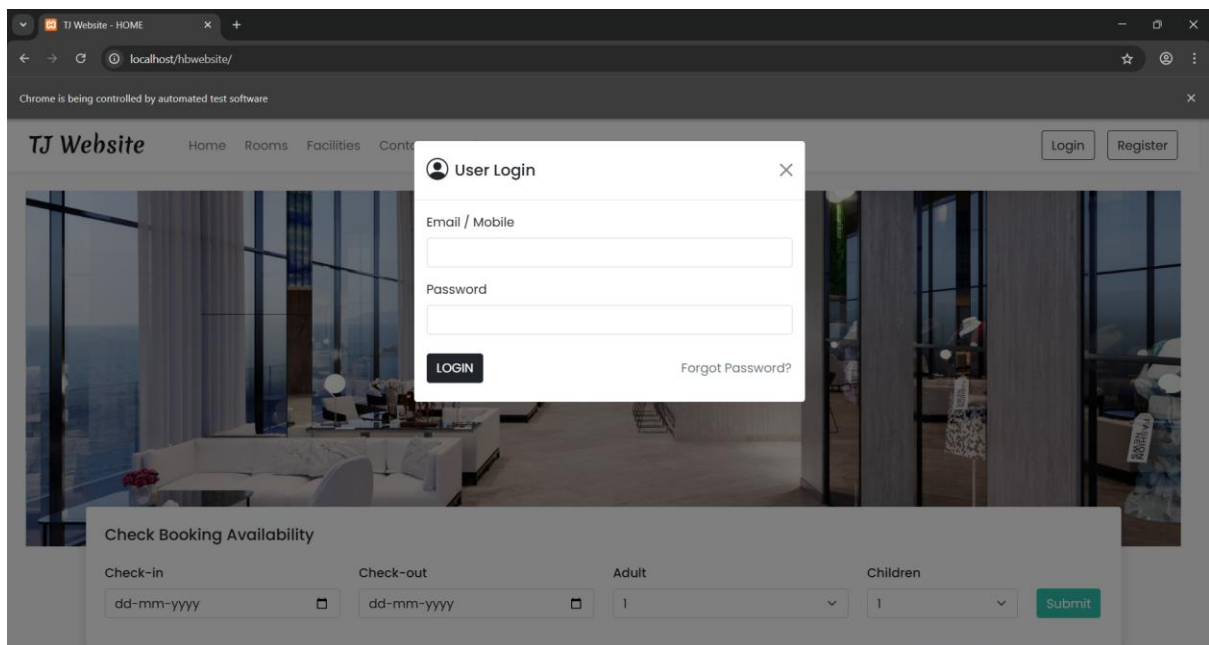
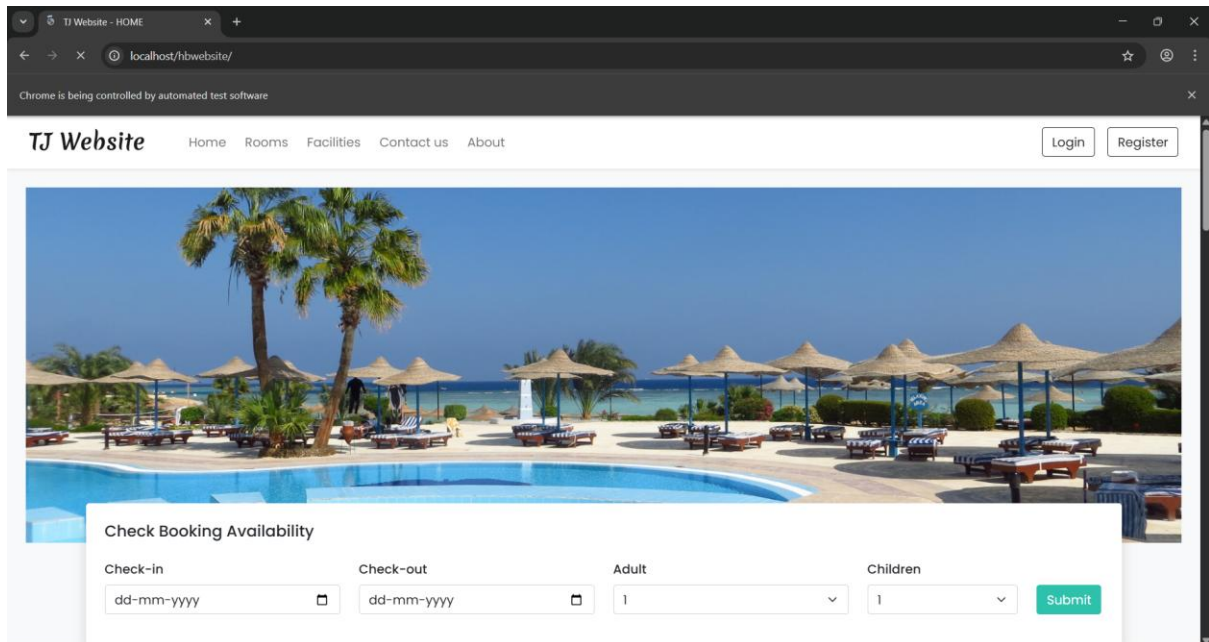
[Done] exited with code=0 in 28.014 seconds

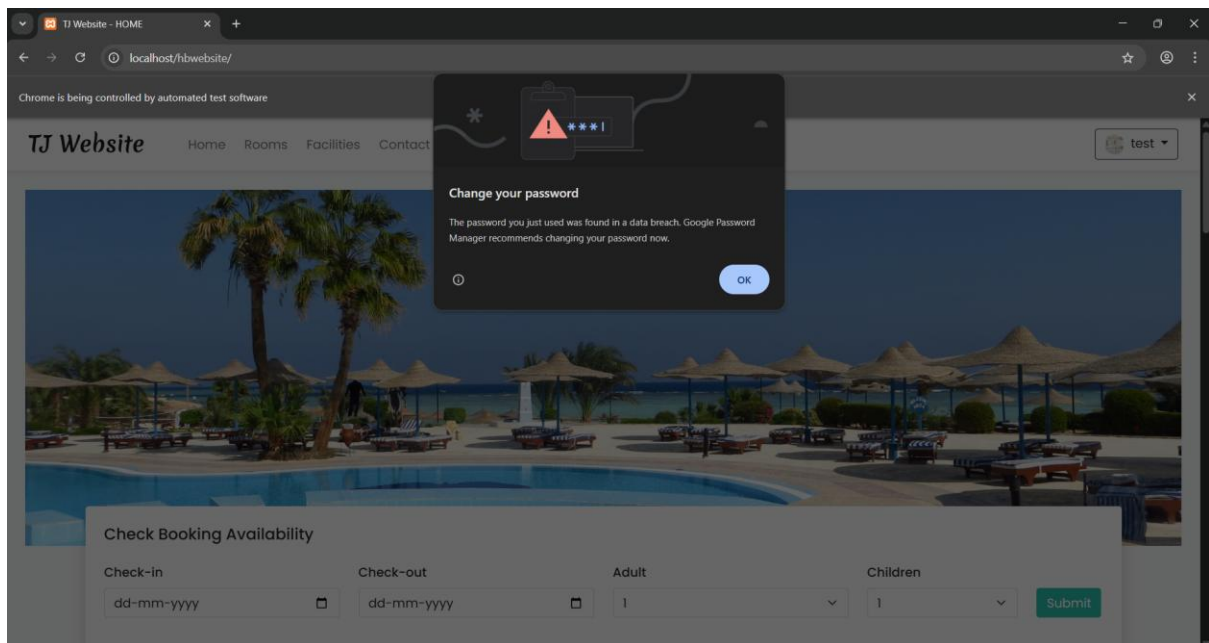
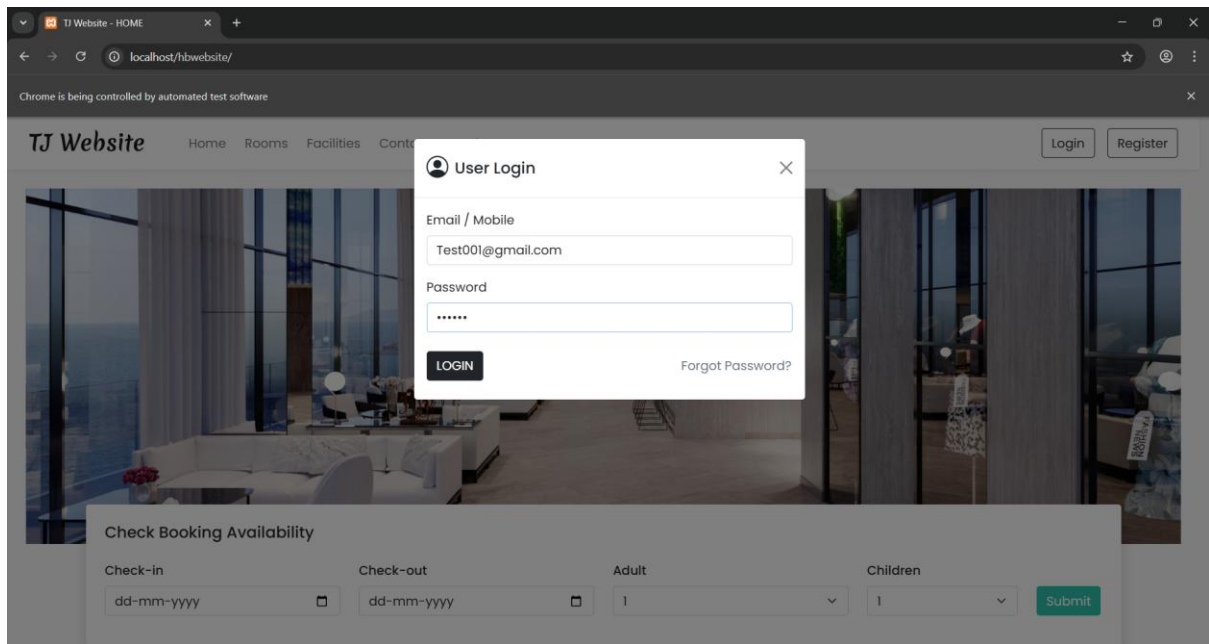
```

6.2 Final Test Execution Output

The console report confirms that the browser automation was successful and the user was authenticated.

- **Description:** CLI output displaying "Browser opened," "Login submitted," and the final "Test Result: LOGIN SUCCESSFUL - PASS" status.





7. CONCLUSION

The implementation of automated system testing successfully validates the **User Authentication** flow of the Hotel Management System. By utilizing **Selenium WebDriver** and **WebDriver Manager**, the project ensures that the frontend interactions correctly trigger the backend logic. With a successful login and session verification, the application is verified as functionally stable for the user-side environment.